UTN FRC | FACULTAD REGIONAL CÓRDOBA

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

BASE DE DATOS I

Segunda Entrega

Curso: 1w3

Profesor: Damian Canovas

Integrantes

412465 – Benjamin Ceballos

412135 - Valentin Caceres

430113 - Tomás Di Marco Tutor

404925 - Bruno Tomas Diomedi

Fecha de Entrega: 15/10/2026

INDICE

ENUNCIADO 2 – VENTA DE ENTRADAS CON BUTACAS ENUMERADAS EN UN COMPLEJO DE CINES 3

DIAGRAMA BASE DE DATOS 4

CONSULTAS 5

CONSULTA BENJAMIN CEBALLOS 5

CONSULTA VALENTIN CACERES 6

CONSULTA BRUNO DIOMEDI 8

CONSULTA TOMAS DI MARCO TUTOR 11

ENUNCIADO 2 – VENTA DE ENTRADAS CON BUTACAS ENUMERADAS EN UN COMPLEJO DE CINES

Un conocido complejo de salas de exhibición de filmes de la Ciudad de Córdoba desea contar con un sistema que registre la compra de tickets por parte de los clientes.

Para ello se debe diseñar una base de datos teniendo en cuenta la información relevada para tal fin: Esta empresa posee varias salas de proyección de distintos tipos (común, vip, 3D, etc.), todas ellas con butacas numeradas.

En el momento en que una persona se acerca a comprar entradas (a una ventanilla o por Internet), se emite un comprobante donde figuran los datos de la compra realizada (película, tipo de sala, función, cantidad de entradas, monto y forma de pago) y además un ticket por cada entrada donde debe constar la sala, la película, día, horario (función) y el nro. de butaca. Una misma persona puede comprar, en un mismo momento varias entradas a distintas películas.

Hay que tener en cuenta, que existen días y horarios promocionales, es decir con descuentos.

Además, se pueden reservar entradas por internet la cual se confirma con el pago de estas. Esta reserva (si no se confirma) vence dos horas antes del inicio de función.

Pueden basarse para el desarrollo de esta base de datos y la posterior programación de los informes una sala conocida por el grupo.

Los reportes posibles para este sistema de información son, por ejemplo:

• Ingresos totales mensuales por día de proyección, o por temporada (vacaciones de invierno, de verano, feriados) o por bloques horarios.

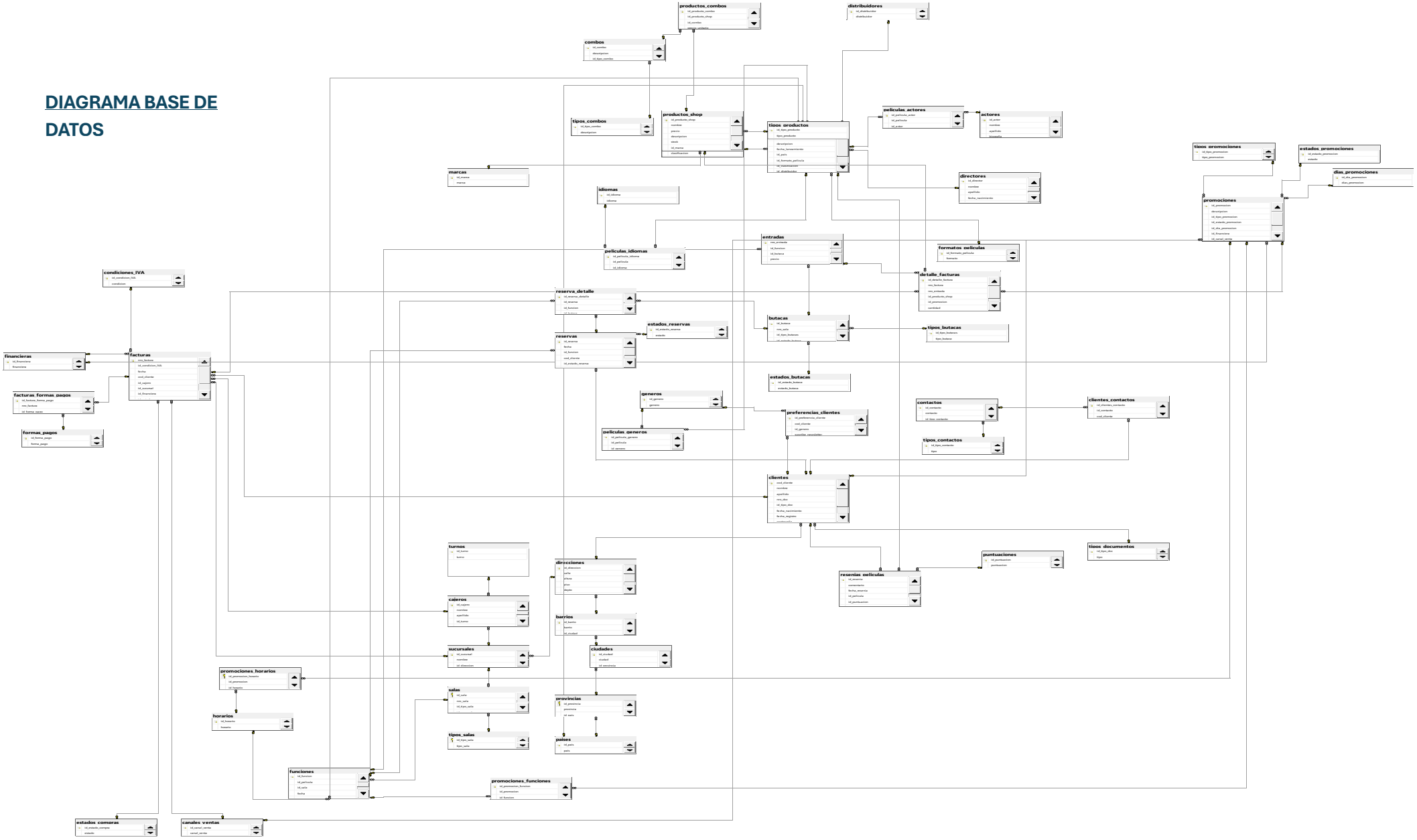
• Tipo de público que asiste en distintos horario o temporadas, tipos de películas vistas, etc.

• Tipo de público (menores, mayores, jubilados) que nunca asistió a una determinada película o tipo de película (terror, acción, animadas, etc.)

• Horarios o películas que no tuvieron ningún asistente

• Películas más vistas.

DIAGRAMA BASE DE DATOS



TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN UTN-FRC

CONSULTAS

Consulta de Benjamin Ceballos

Este procedimiento genera un informe completo y detallado sobre las reservas del mes actual, clasificadas según su estado (confirmadas, canceladas, pendientes, etc.). Permite visualizar de manera rápida el desempeño mensual de cada estado de reserva, cuántas reservas se realizaron, cuánto dinero se recaudó, el promedio por operación, y cuántos clientes distintos participaron durante el mes, además, ayuda a detectar picos de demanda.

```
CREATE PROCEDURE pa_reservas_por_estado_mes_actual_
AS
BEGIN
DECLARE @mesActual INT = MONTH(GETDATE());
DECLARE @anioActual INT = YEAR(GETDATE());
SELECT
er.estado AS EstadoReserva,
COUNT(r.id_reserva) AS CantidadReservas,
SUM(r.precio_reserva) AS TotalRecaudado,
AVG(r.precio_reserva) AS PromedioPorReserva,
MIN(r.fecha) AS PrimeraReserva,
MAX(r.fecha) AS UltimaReserva,
COUNT(DISTINCT cod_cliente) AS ClientesDistintos
FROM reservas r
WHERE MONTH(r.fecha) = @mesActual
AND YEAR(r.fecha) = @anioActual
AND r.precio_reserva > 0
GROUP BY er.estado
HAVING COUNT(r.id_reserva) > 0
```

```
ORDER BY TotalRecaudado DESC, CantidadReservas DESC;
END;
```

Consulta de Valentin Caceres:

Este procedimiento almacenado sirve para saber la recaudación de las películas en un rango de fechas determinado. Permite ingresar una fecha de inicio, una fecha de fin y el código de película. Cuando se ejecuta devuelve la cantidad total de entradas vendidas, la recaudación total obtenida y con la subconsulta se calcula el promedio general de las películas en ese periodo de tiempo.

```
CREATE PROCEDURE SP_Recaudacion_Por_Pelicula
  @FechaDesde DATE,
@FechaHasta DATE,
@IdPelicula INT = NULL
AS
BEGIN
SELECT
p.id_pelicula,
p.titulo Pelicula,
SUM(df.cantidad_entradas) TotalEntradasVendidas,
SUM(CAST(df.cantidad_entradas AS DECIMAL(18,2)) * df.precio) TotalRecaudado,
MIN(f.fecha) PrimeraFuncion,
MAX(f.fecha) UltimaFuncion
FROM (
SELECT
df.nro_entrada,
df.nro_factura,
df.cantidad_entradas,
df.precio
```

```
FROM
facturas fa
JOIN detalle_facturas df ON df.nro_factura = fa.nro_factura
WHERE CONVERT(DATE, fa.fecha) BETWEEN @FechaDesde AND @FechaHasta
)
df
 JOIN entradas e ON e.nro_entrada = df.nro_entrada
JOIN funciones f ON f.id_funcion = e.id_funcion
JOIN películas p ON p.id_pelicula = f.id_pelicula
WHERE @IdPelicula IS NULL OR p.id_pelicula = @IdPelicula
GROUP BY p.id_pelicula, p.titulo
ORDER BY TotalRecaudado DESC;
END;
```

Consulta de Bruno Diomedi:

Este SP te arma una lista de los mejores clientes para después mandarles promociones o premios, juntando toda la informacion de facturas, entradas, funciones y películas para ver quiénes compraron más y gastaron más. Muestra el nombre del cliente, cuántas películas distintas vio, cuántas entradas compró, cuánto dinero gastó en total, las fechas de su primera y última compra y un contacto si tiene. Se puede filtrar por fechas, por una película en particular o por cantidad mínima de películas vistas, y también podés pedir que te muestre solo los primeros que más gastaron.

```
CREATE PROCEDURE SP_ClientesFrecuentesPromos

@FechaDesde     DATE      = NULL,

@FechaHasta     DATE      = NULL,

@IdPelicula     INT       = NULL,

@MinPeliculas   INT       = 0,

@Top            INT       = NULL

AS

BEGIN

IF @Top IS NULL

 BEGIN

SELECT

c.cod_cliente,

 c.nombre  NombreCliente,

c.apellido  ApellidoCliente,

COUNT(DISTINCT f.id_pelicula) PeliculasDistintas,

COUNT(e.nro_entrada) TotalEntradas,

SUM(df.cantidad_entradas * df.precio) TotalGastado,

MIN(CONVERT(date, fa.fecha)) PrimeraCompra,

MAX(CONVERT(date, fa.fecha)) UltimaCompra,

MAX(ct.contacto) Contacto

FROM facturas fa
```

```
JOIN detalle_facturas df ON df.nro_factura = fa.nro_factura
JOIN entradas e ON e.nro_entrada = df.nro_entrada
 JOIN funciones f ON f.id_funcion = e.id_funcion
    JOIN peliculas p ON p.id_pelicula = f.id_pelicula
JOIN clientes c ON c.cod_cliente = fa.cod_cliente
LEFT JOIN clientes_contactos cc ON cc.cod_cliente = c.cod_cliente
LEFT JOIN contactos ct ON ct.id_contacto = cc.id_contacto
 WHERE (@FechaDesde IS NULL OR CONVERT(date, fa.fecha) >= @FechaDesde)
AND (@FechaHasta IS NULL OR CONVERT(date, fa.fecha) <= @FechaHasta)
AND (@IdPelicula IS NULL OR f.id_pelicula = @IdPelicula)
GROUP BY c.cod_cliente, c.nombre, c.apellido
HAVING COUNT(DISTINCT f.id_pelicula) >= @MinPeliculas
ORDER BY TotalGastado DESC;
END
ELSE
BEGIN
SELECT TOP(@Top)
c.cod_cliente,
 c.nombre NombreCliente,
 c.apellido ApellidoCliente,
 COUNT(DISTINCT f.id_pelicula) PeliculasDistintas,
COUNT(e.nro_entrada) TotalEntradas,
SUM(df.cantidad_entradas * df.precio) TotalGastado,
MIN(CONVERT(date, fa.fecha)) PrimeraCompra,
MAX(CONVERT(date, fa.fecha)) UltimaCompra,
MAX(ct.contacto) Contacto
FROM facturas fa
JOIN detalle_facturas df ON df.nro_factura = fa.nro_factura
```

```
 JOIN entradas e ON e.nro_entrada = df.nro_entrada

JOIN funciones f ON f.id_funcion = e.id_funcion

JOIN peliculas p ON p.id_pelicula = f.id_pelicula

JOIN clientes c ON c.cod_cliente = fa.cod_cliente

LEFT JOIN clientes_contactos cc ON cc.cod_cliente = c.cod_cliente

LEFT JOIN contactos ct ON ct.id_contacto = cc.id_contacto

WHERE (@FechaDesde IS NULL OR CONVERT(date, fa.fecha) >= @FechaDesde)
AND (@FechaHasta IS NULL OR CONVERT(date, fa.fecha) <= @FechaHasta)
AND (@IdPelicula IS NULL OR f.id_pelicula = @IdPelicula)

GROUP BY c.cod_cliente, c.nombre, c.apellido

HAVING COUNT(DISTINCT f.id_pelicula) >= @MinPeliculas

ORDER BY TotalGastado DESC;

END

END
```

Consulta de Tomar Di Marco Tutor:

Este SP muestra las funciones que hay disponibles en el cine según el género de la película, que se encuentre dentro de un rango de fechas determinado, y que muestre solo los datos de las películas que ya han sido estrenadas. También incluye la cantidad de funciones encontradas y la cantidad de formatos distintos.

```
create procedure sp_funciones

@Genero varchar(50),

@FechaDesde date,

@FechaHasta date

as

begin

select p.titulo, g.genero, f.fecha, h.horario, s.nro_sala, ts.tipo_sala, fp.formato,
d.nombre + ' ' + d.apellido AS Director

from funciones f

join peliculas p on f.id_pelicula = p.id_pelicula

join peliculas_generos pg on p.id_pelicula = pg.id_pelicula

join generos g on pg.id_genero = g.id_genero

join horarios h on f.id_horario = h.id_horario

join salas s on f.id_sala = s.id_sala join tipos_salas ts on s.id_tipo_sala =
ts.id_tipo_sala

join formatos_peliculas fp on p.id_formato_pelicula = fp.id_formato_pelicula

join directores d on p.id_director = d.id_director

where g.genero like '%' + @Genero + '%'

and f.fecha between @FechaDesde and @FechaHasta

and p.fecha_lanzamiento < getdate()

order by f.fecha, h.horario;

select count(*) 'Total de funciones', count(distinct fp.id_formato_pelicula)
'Cantidad de formatos'

from funciones f
```

```
join peliculas p on f.id_pelicula = p.id_pelicula

join peliculas_generos pg on p.id_pelicula = pg.id_pelicula

join generos g on pg.id_genero = g.id_genero

join salas s on f.id_sala = s.id_sala

join formatos_peliculas fp on p.id_formato_pelicula = fp.id_formato_pelicula
where g.genero like '%' + @Genero + '%'

and f.fecha between @FechaDesde and @FechaHasta

and p.fecha_lanzamiento < getdate();

end;
```